

DevOps, Software Evolution and Software Maintenance

Course code: KSDSESM1KU

Exam assignment by:

Erle Sognnæs: s25128@itu.dk

Jihyeon Yum: s25126@itu.dk

Margrét Edda Friðgeirsdóttir: s25117@itu.dk

Mathias Anakin Alm: matal@itu.dk

Table of content

- **1. Introduction**
- **2. System's Perspective**
 - *2.1 System Architecture*
 - *2.2 System Design*
 - *2.3 Dependencies*
 - *2.4 Current State*
- **3. Process Perspective**
 - *3.1 Infrastructure as Code (IaC)*
 - *3.2 CI/CD Pipeline*
 - *3.3 Monitoring*
 - *3.4 Logging*
 - *3.5 Security Hardening*
 - *3.6 Availability and Scaling*
- **4. Reflection Perspective**
- **5. Use of Generative AI**

Contribution Distribution

Member	Main contributions
Erle Sognnæs	CI/CD, Architecture, System Design
Jihyeon Yum	Availability and scaling, Dependencies, Security hardening
Margrét Edda Friðgeirsdóttir	Monitoring, reflections, Current state
Mathias Anakin Alm	Infrastructure, Logging, Use of Generative AI

1) Introduction

This report describes the design, evolution, and operation of our ITU-MiniTwit system. Our group's goal was to upgrade the old application into a robust, scalable application by using DevOps principles.

We implemented Infrastructure as Code (IaC) using Vagrant to ensure our environments stay identical. We then implemented Prometheus and Grafana for observing the system through monitoring and logging. We then deployed a Nginx load balancer integrated with Keepalived to achieve high availability, to automatically handle server failures. The development lifecycle is managed through a CI/CD pipeline, which automates testing and deployment processes.

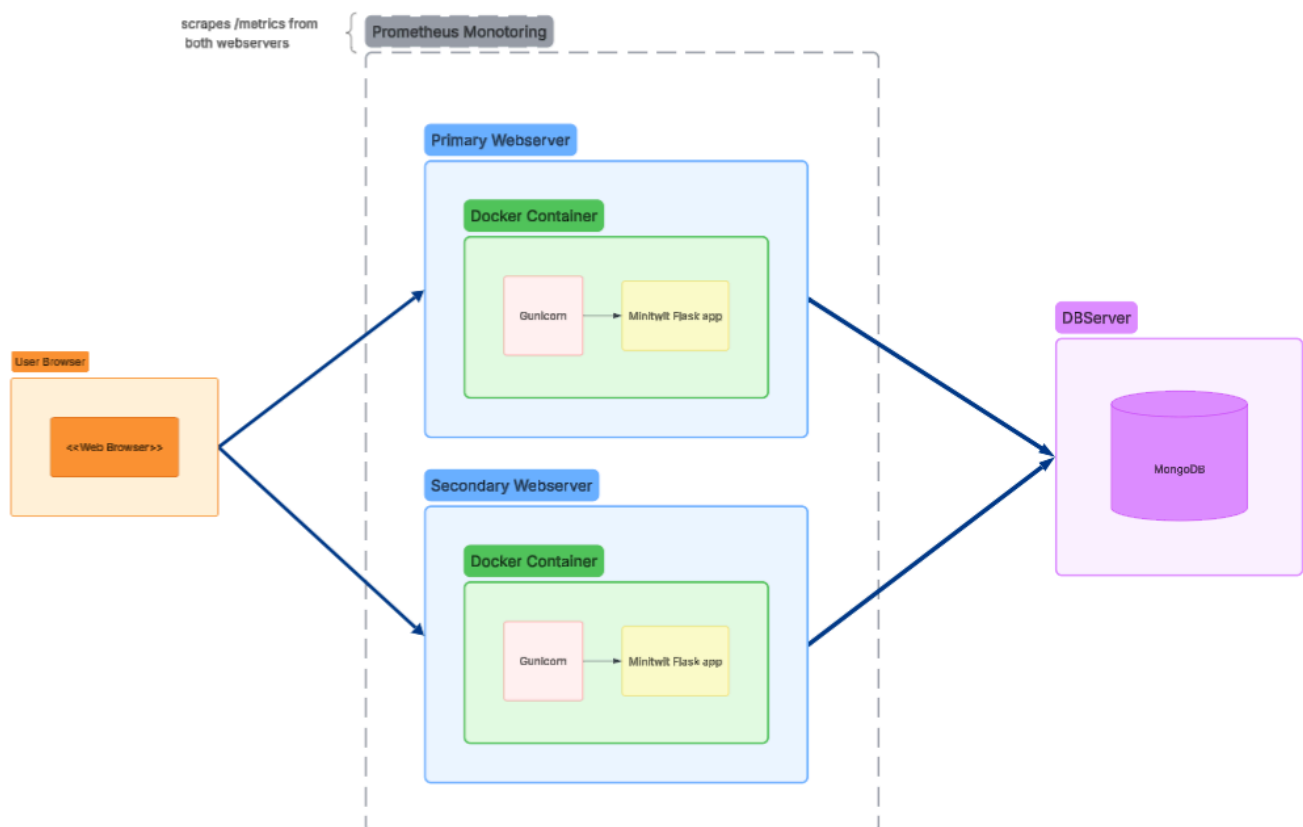
2) System's Perspective

This section explains the layout of our ITU-MiniTwit, from physical servers down to the code. By breaking the system into separate pieces, we make sure our environments stay consistent, prevent isolated bugs from crashing the whole application, and make it easy to add servers when traffic grows.

2.1) System Architecture

The three-tier architecture we implemented splits our system into user interface, core application logic, and database storage.

Allocation View



The diagram shows how we implemented the three-tier architecture. The User Interface Layer consists of the user's web browser, which displays the MiniTwit UI. The Application Logic Layer has two identical web servers running in parallel, each implemented with a Docker, for consistency across different servers. Gunicorn hosts the MiniTwit Flask application, handles incoming HTTP requests and passes them to our main application. The Database Layer includes MongoDB which communicates with both servers, and stores all application data. Additionally, Prometheus monitors and scrapes performance metrics from both web servers.

Note: To keep our system horizontally scalable, our logic tier is stateless. The servers run independently and use the Flask-PyMongo client as bridge to fetch and update data from the shared database.

2.2) System Design

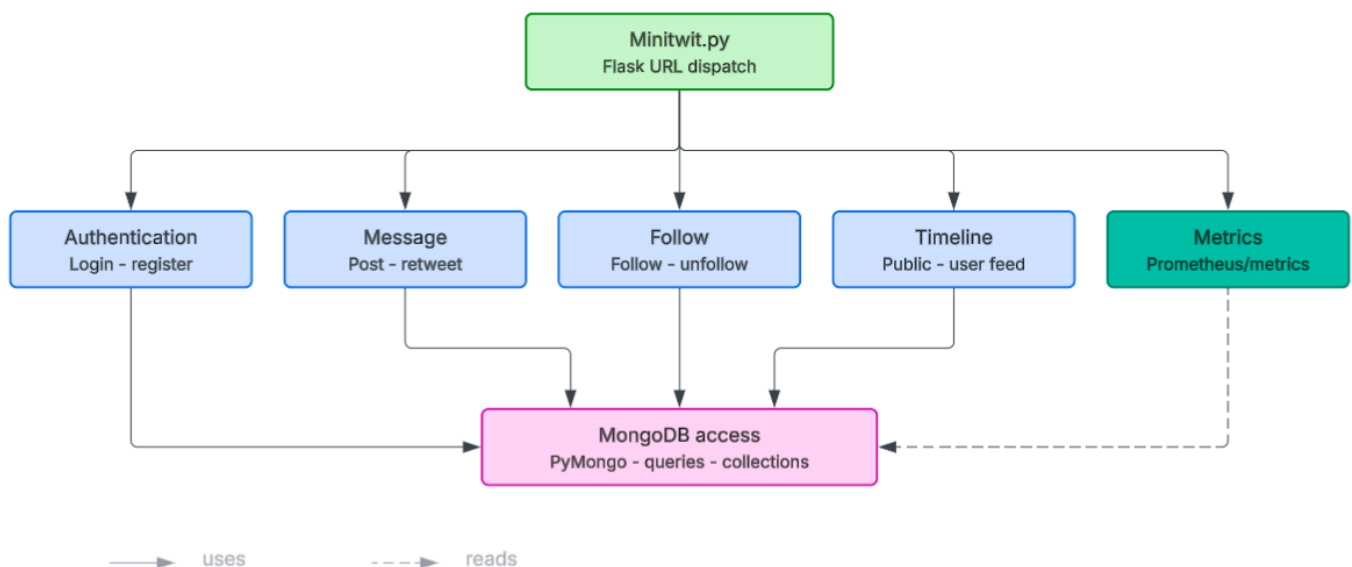
We used Flask, a lightweight Python web framework, to build our application because it provides a simple way to manage routing, user authentication, and database integration.

Our Flask setup is broken down into two main responsibilities:

- Frontend Rendering: Flask uses the Jinja2 templating engine to generate the HTML pages users see, dynamically injecting Python data like usernames and tweets directly into the frontend.
- Backend Operations: Flask handles all core user actions (logins, tweets, and follow requests). It relies on its built-in utility library, Werkzeug, to securely hash user passwords and manage secure sessions.

To talk to our database, Flask routes these backend requests through Flask-PyMongo. This extension acts as our direct bridge to the MongoDB server, utilizing the official PyMongo driver to execute queries and manage data collections.

Module View

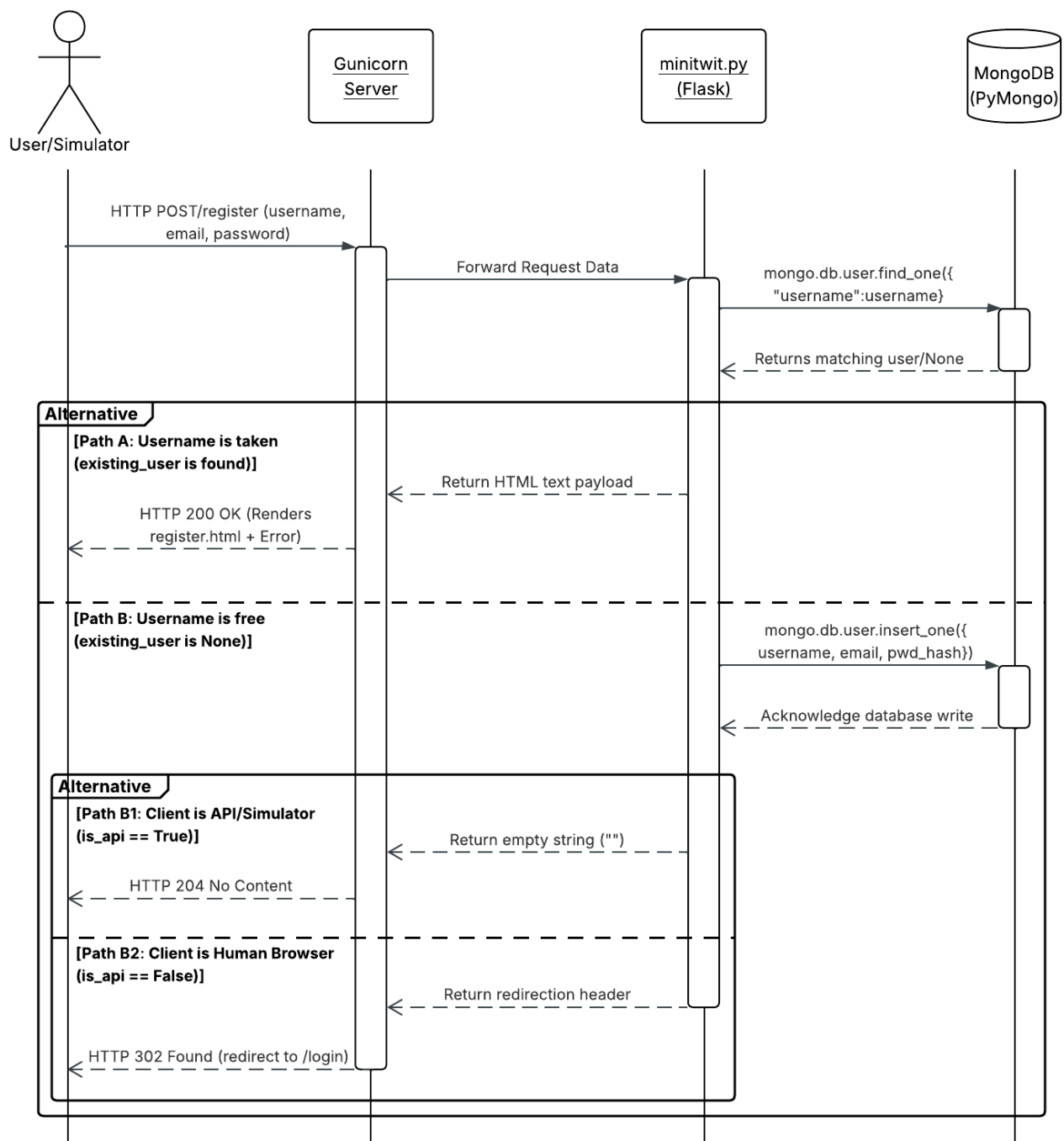


As shown in the diagram, Minitwit.py is our core application code acts as the entry point, routing incoming web traffic to specific Python features:

- Authentication handles user registration and login sessions.
- Message manages posting new tweets and retweeting.
- Follow controls following and unfollowing other users.
- Timeline builds the public and personal user feeds.
- Metrics tracks performance data for Prometheus to scrape.

All these features communicate with a shared MongoDB access layer using PyMongo. This layer handles all database queries, keeping our data operations isolated from our application logic features.

Component and Connector view



This sequence diagram shows the lifecycle of registering a user. The process starts when a client, either User or Simulator, sends in their information. Gunicorn catches the data and forwards it to our Flask application. Flask queries MongoDB with `find_one` to look for the username in the database. MongoDB returns if the username is taken or not. Here the logic splits into alternative branches.

- Path A: username is taken. In this case, execution stops and an error message, "The username is already taken", is sent through the system and shown to the user.
- Path B: username is free. In this case Flask hashes the password and sends the users information to the database through `insert_one`.

After successful registration the execution enters the inner alternative block

- Path B1: if client is a Simulator. The system returns an empty string and a HTTP 204 to register success, but not use unnecessary internet speed with redirecting to login.

- Path B2: if client is a Human user. Flask activates the redirect function, which sends the user to the login page.

2.3) Dependencies

The system is implemented in Python using the Flask web framework and MongoDB via PyMongo. Passwords are handled with Werkzeug security utilities. Monitoring is supported through Prometheus using `prometheus_client` and `prometheus_flask_exporter`. The application is containerized with Docker and orchestrated through Docker Compose. Development and version control are handled with Git and GitHub, while CI/CD is handled through GitHub Actions. Code quality/security analysis is configured through Sonar using `sonar-project.properties`. Our deployment dependencies are illustrated in our Allocation view diagram.

2.4) Current state

Area	Current state
Web framework	Flask
Database	MongoDB via Flask-PyMongo/PyMongo
Testing	Pytest and Selenium
Static analysis	Flake8 and SonarCloud
Formatting	Black
Deployment	Docker, Docker Compose, Gunicorn
Monitoring	Prometheus, Grafana, Loki, cAdvisor
Main improvement	Further decouple application and database logic

The production setup currently remains stable across both web nodes and supports concurrent traffic from our automated simulator. Automated formatting via Black and linting via Flake8 improved our code maintainability and consistency. However, parts of `minitwit.py` still directly access MongoDB instead of using a fully isolated data-access layer, creating tighter coupling between application logic and the database. A remaining improvement goal is to fully decouple this integration to reduce the impact of future schema or infrastructure changes.

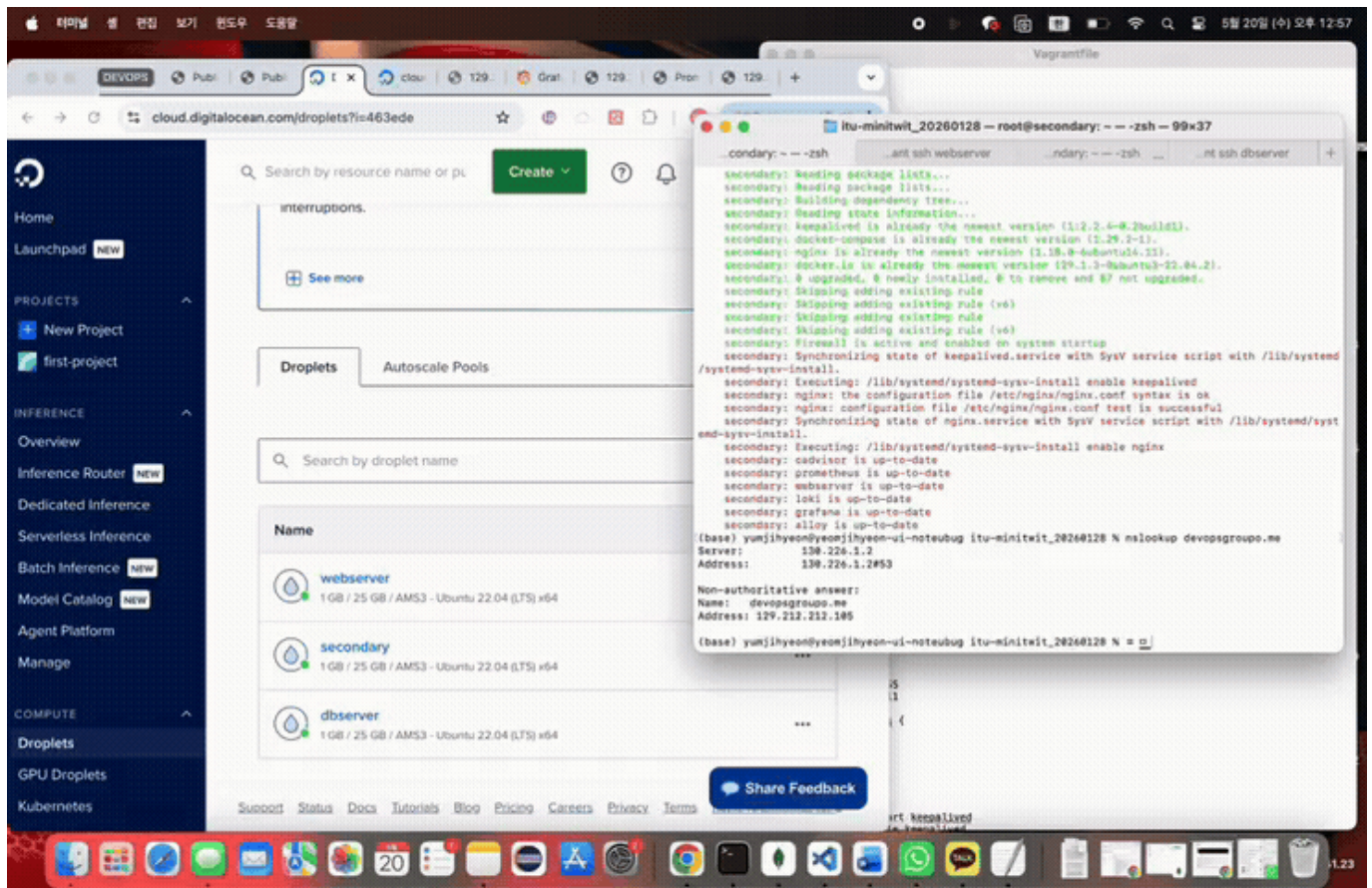
3) Process perspective

The Process Perspective highlights the automated lifecycles that construct, validate, provision, and maintain our systems.

3.1) Infrastructure as Code (IaC)

To ensure our environments are reproducible and resilient against hardware failures, our state configuration is managed as code.

IaC in Action

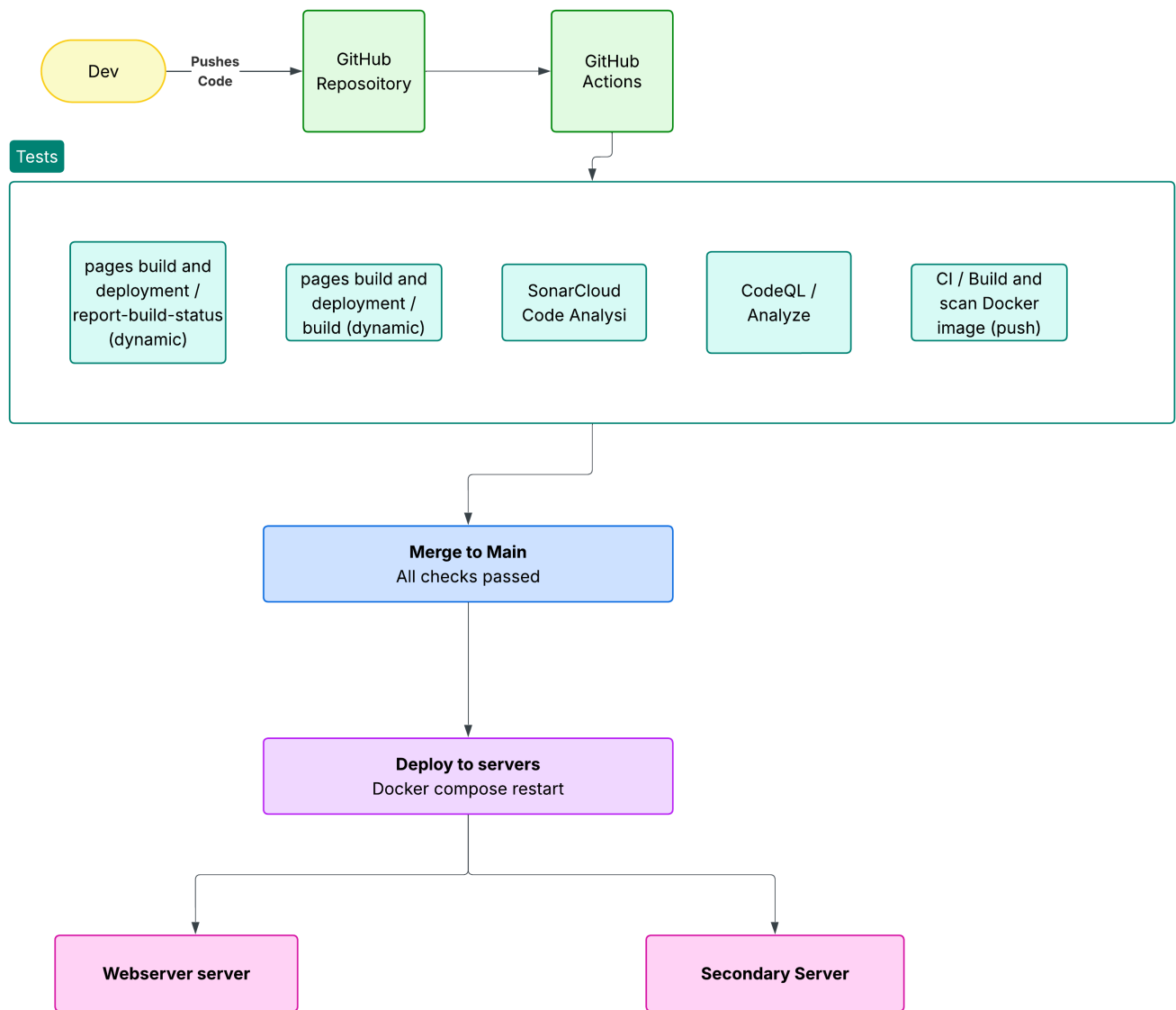


We implemented Infrastructure as Code (IaC) via Vagrant and the DigitalOcean provider to ensure our environment is reproducible. Our infrastructure is orchestrated via a single Vagrantfile, which automates the creation and configuration of three virtual droplets: dbserver, webserver (Primary), and secondary.

The Vagrantfile includes shell provisioning scripts that automate the installation of core dependencies, including Docker, Docker Compose, and Nginx. It also handles complex network configurations, such as setting up Keepalived on both web servers to manage a Virtual IP (VIP) for automated failover. Furthermore, the IaC layer handles the preparation required for Observability in a multi-process environment. This includes automated creation of shared memory directories (e.g., `/app/prometheus_metrics`) with restricted permissions (755) to enable consistent metric collection via Unicorn workers. By executing a single command `vagrant up`, the entire infrastructure is built from scratch in minutes, eliminating manual configuration errors and ensuring high system reliability. Our provisioning setup builds up our environment from a blank slate. As captured in the GIF above, our scripts handle downloading dependency layers, initializing the Docker engines, structuring the virtual container bridges, and linking our web nodes cleanly to our database servers minimizing configuration drift between staging and production nodes.

3.2) CI/CD Pipeline

Our software deployment loop runs automatically whenever updates are committed to source control.



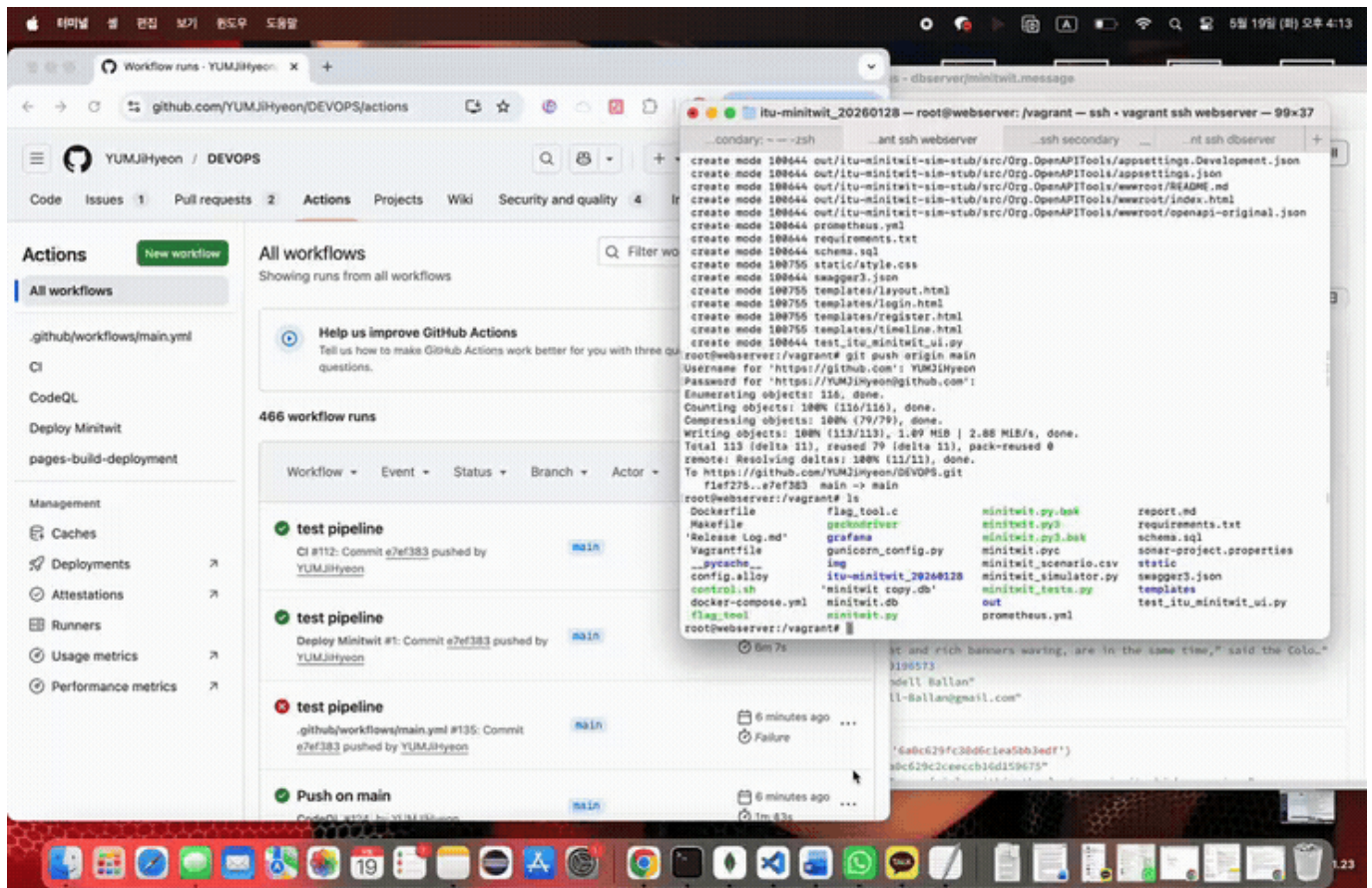
As shown above, our pipeline starts when code is pushed onto our Github repository. Github Actions then runs tests automatically and in parallel.

The pipeline includes automated build processes, dynamic page deployments, SonarCloud static code analysis, CodeQL security analysis, and Docker image build and scan operations. These automated checks help ensure code quality, security, and deployment consistency before changes are merged into the main branch.

Once all checks have passed, the application is then deployed to both the primary web server and secondary web server using Docker Compose restart procedures.

This CI/CD pipeline automates much of the development and deployment workflow, reducing manual configuration errors and supporting continuous integration and continuous deployment practices.

CI/CD in Action



3.3) Monitoring

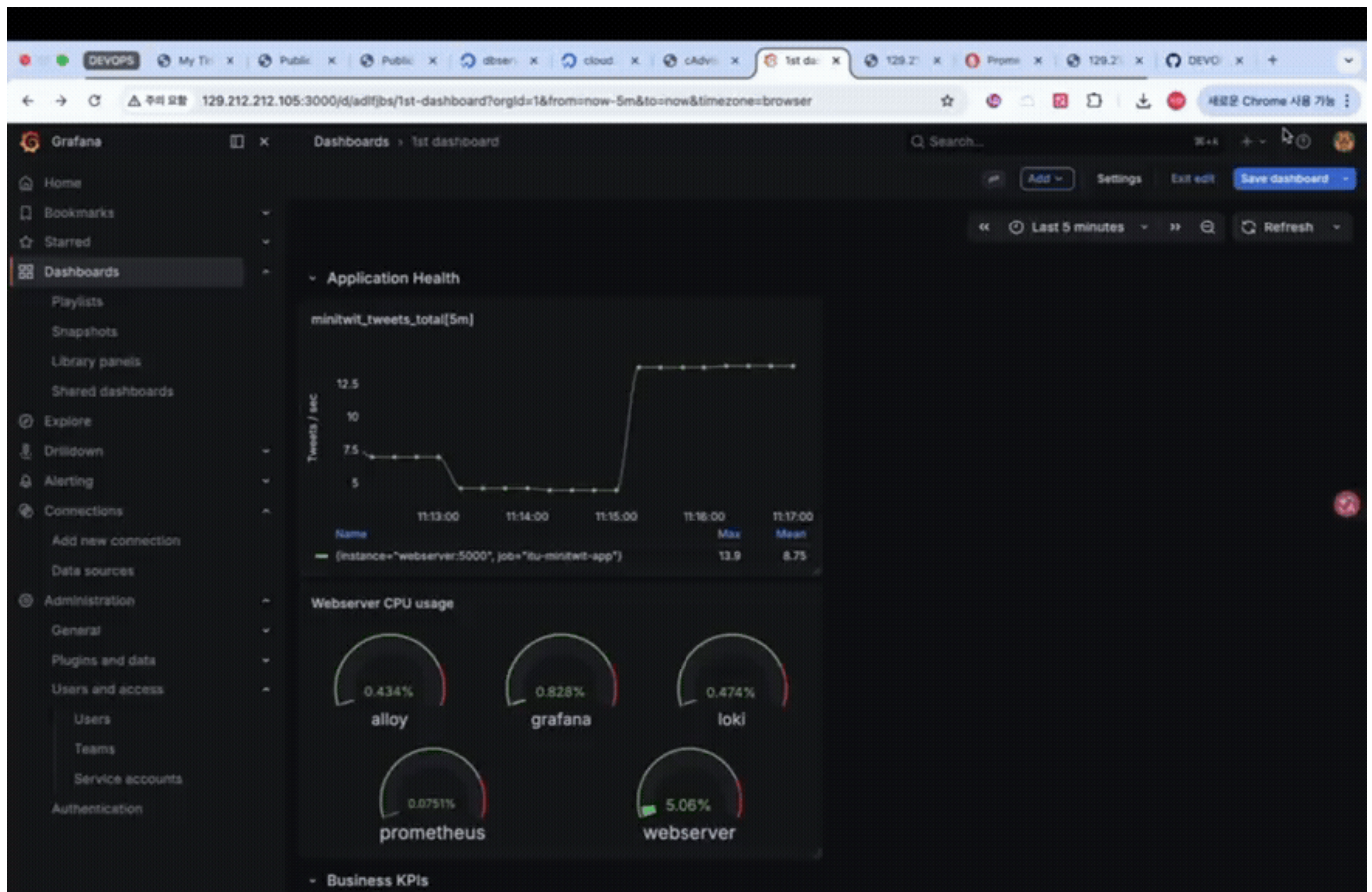
We monitored our system using a Prometheus and Grafana stack to ensure infrastructure health, application performance, and business visibility. Resource consumption was monitored through cAdvisor, collecting real-time data from Docker containers. We tracked CPU usage per container helping us to identify bottlenecks in services like MongoDB or the web server.

Application health was monitored via prometheus-flask-exporter in minitwit.py. We tracked flask_http_request_total across several HTTP response types, including successful requests (200), redirects (302), missing routes (404), and unsupported methods (405). This also helped detect suspicious traffic patterns from vulnerability scanners such as BXJZ, PROPFIND, and HIAS.

Business KPIs were monitored through PromQL queries. Total registered users were tracked using max(minitwit_users_total), where the max function ensured consistent values across Unicorn worker processes. User activity was visualized through tweet frequency providing insights into real-time system engagement.

To handle Unicorn's multiprocess architecture, we implemented UnicornInternalPrometheusMetrics and configured the PROMETHEUS_MULTIPROC_DIR, ensuring metrics from multiple workers were aggregated into a consistent shared state.

Monitoring Dashboard in Action

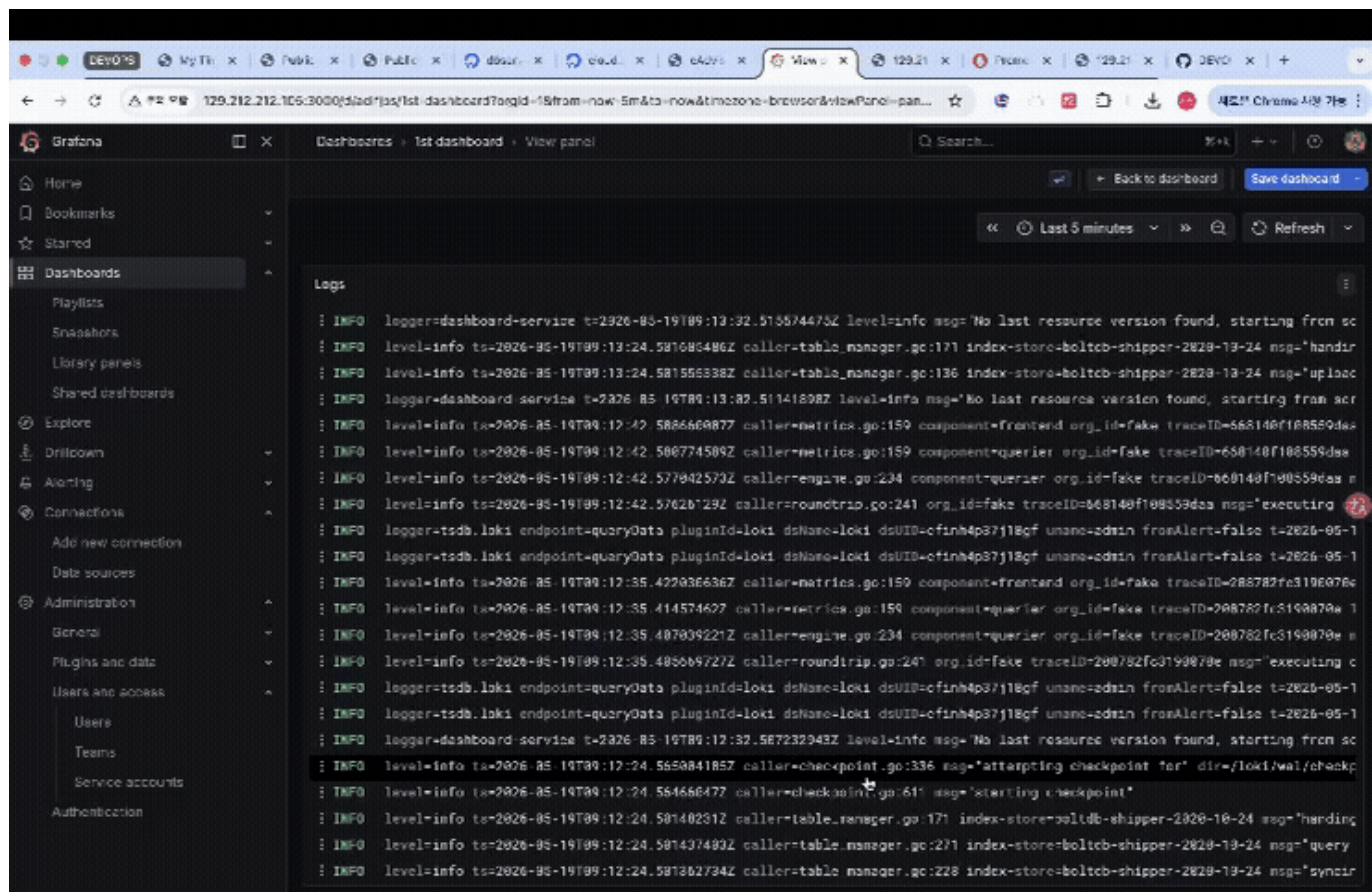


3.4) Logging

Our system logged used Grafana Loki for centralized aggregation, collected via Alloy. While Prometheus monitored numerical metrics, Loki captured the textual content of HTTP requests, including methods, paths, and client details.

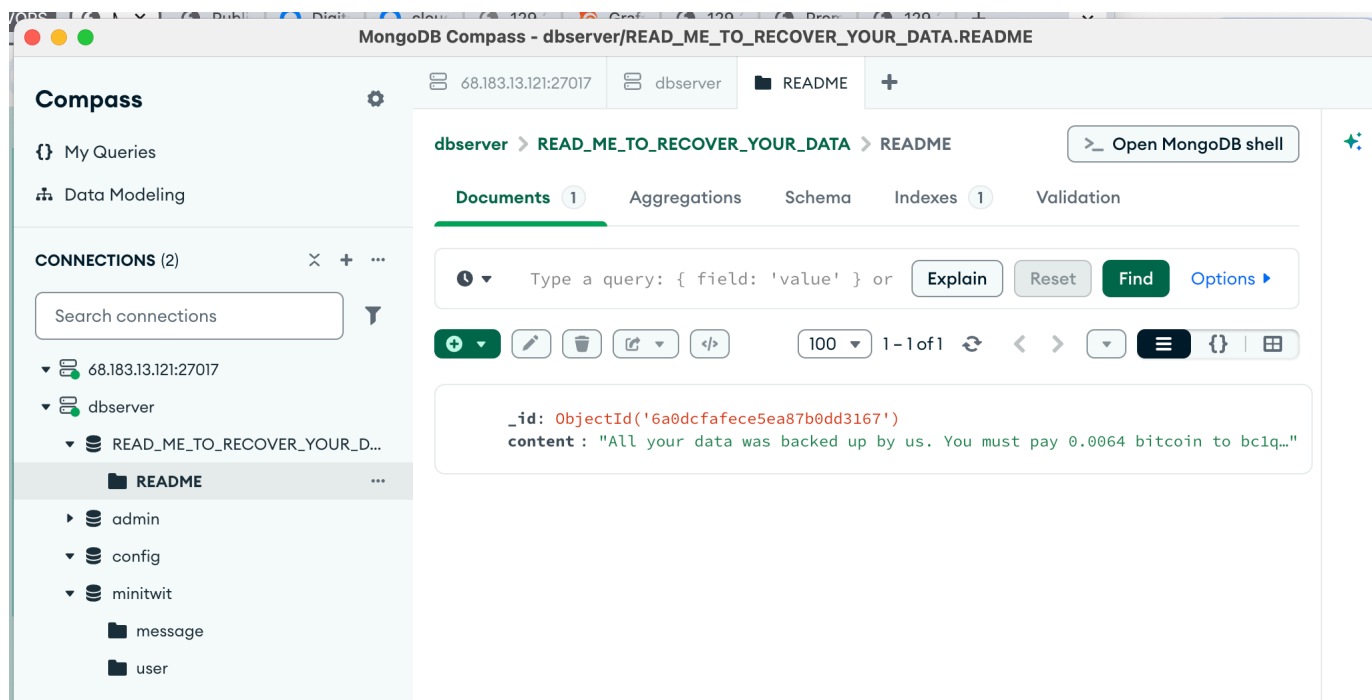
We recorded application-level debug messages, such as user registrations and tweet attempts, alongside critical system errors like Gunicorn worker timeouts and Python tracebacks. Correlating Prometheus metric spikes with Loki log entries significantly reduced our mean time to resolution (MTTR) during incidents.

Logging Dashboards in Action



As shown above, our system continuously logs directly into Grafana Loki. Exceptions, stack traces, and suspicious scanner requests became searchable in real time. Tenabling faster debugging and reducing the need for manual shell access to production servers.

3.5) Security hardening



We security hardened our deployment by making sure production host data boundaries are fully isolated from our public code history:

- **Environment Files (.env):** We use `.env` files to store our database passwords and secret keys locally on the server. This prevents it from accidentally being pushed to GitHub.
- **Docker Ignore (.dockerignore):** Acts like a filter during builds. It makes sure that junk files, local test logs, and temporary development data don't get accidentally bundled into our live production containers.
- **Runtime Scans:** We set up CodeQL inside our GitHub pipeline to act as an automated security check. Every time someone pushes a new pull request, the pipeline automatically scans the code for security bugs, unescaped database queries, or leaked keys before we are allowed to merge it.

As shown in the picture, a hacker bot managed to enter our database early in the course, when we hadn't been able to make it secure yet. We realized when the simulator stopped, and this made for big confusion.

3.6) Availability and scaling

Our system successfully handed over the MASTER role by utilizing Keepalived and a custom script to call the DigitalOcean API. However, automated state monitoring and failover via real-time signals between nodes were not fully achievable. We identified that the DigitalOcean Cloud Firewall blocked the VRRP protocol, preventing the secondary node from receiving the primary node's heartbeat signals. While an API-based manual transition was functional, these network-level constraints limited the implementation of a fully automated and mutually monitored failover mechanism.

4) Reflection Perspective

One major issue during the evolution of our system was migrating from the SQLite-based Flask application to a MongoDB-backed. The original system used a local database file. That made it simple but Unfit for a distributed setup. The refactored version used flask_pymongo and MONGO_URI, making the database external and configurable. That also introduced challenges like networking, firewall rules, and database availability. The system's evolution is reflected in commits such as 6598b74 (Docker hardening), 0bb4adf (Grafana monitoring fixes), and c011bde (secondary webserver and scaling support).

The scope of the refactoring is summarised below:

First app	Second app
Uses SQLite	Uses MongoDB
Local DB file	External DB server
Simple Flask app	Production-style app
No metrics	Prometheus/Grafana metrics
No API routes	Has API endpoints
Runs standalone	Designed for Docker/Gunicorn
No env vars	Uses <code>.env</code> + <code>MONGO_URI</code>

The biggest technical challenges we faced were related to high availability, distributed infrastructure, and operational stability. One of the most difficult parts was introducing a secondary webserver and making failover between servers work reliably. We attempted to scale the system toward a high-availability architecture using multiple webserver, nginx reverse proxying, Docker containers, and keepalived for virtual IP

failover. While the architecture worked, ensuring that both servers remained synchronized and operational after redeployments proved difficult.

A recurring issue was that Grafana monitoring would initially function correctly after deployment, but later stop receiving data without any direct changes being made to the system. Although we never fully identified the exact cause, we suspect the failures were related to resource limitations, monitoring load, container restarts, or networking inconsistencies between services. Also nginx reverse proxy configuration sometimes pointed to incorrect services such as Grafana or cAdvisor instead of the Flask application container. This helped us realize that monitoring systems themselves must also be treated as production infrastructure. They require persistent configuration, stable provisioning, and operational maintenance. In this project, success depended on Docker, Nginx, MongoDB, firewall rules, Vagrant, keepalived, monitoring, and HTTPS all functioning simultaneously. This significantly changed our development workflow. Much of our time was spent debugging infrastructure interactions rather than only application code. Our process became highly iterative:

- Deploy infrastructure
- Observe failures
- Inspect logs and metrics
- Fix configuration issues
- Encode the fixes into provisioning scripts

Compared to previous projects, this work was much more operationally focused and aligned more closely with DevOps practices. Unlike prior projects where infrastructure was set up once and left alone, here provisioning scripts were updated alongside application code, making deployment a continuous concern. However, our process had a significant gap: unlike many other groups, we did not inherit a previous semester's Chirp project and instead worked from the teacher-provided legacy MiniTwit codebase. Although a new group member later had access to a prior project, this happened several weeks into the course, and we concluded that switching codebases at that stage was too disruptive. As a result, a large amount of time was spent building infrastructure and adapting legacy code before we could focus on stable iteration and release routines. Despite this, the project gave us practical experience with Infrastructure as Code, distributed systems debugging, observability, and high-availability infrastructure, while also highlighting how much operational complexity a greenfield distributed system introduces even before feature development begins.

5) Use of Generative AI

We have used Gen AI to help identify the cause of cryptic error messages, a lot of this is from trying to deploy and some cryptic error message gets sent back. Using AI for error messages helped us when working with packages we don't have experience in. We have also used AI for vibe coding, this makes our workflow center more on fixing errors and making design decisions. We used AI for helping our report structure. We used ChatGPT, Claude and Gemini for error finding and vibe coding, we used Gemini and ChatGPT for our report structure.